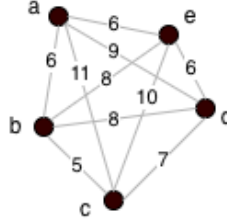


1. Local Search (1.5 pts)

Link cables between a set of servers S to integrate all servers to a single network (each server can reach any other through a path), while keeping the lowest cost for linking cables. The cost of linking any two servers $a, b \in S$ is given by a function $c(a, b)$.



1. Formulate this problem as a local search problem: define (1) the feature representation of a candidate solution; and (2) the loss function of a candidate solution.
2. Apply greedy descent on this problem, write down the resulting candidate solution for the first three iterations.

Solution:

1. (1) We can define a binary feature for each edge to denote whether to choose an edge. Since there are 10 edges, the feature representation would be a vector of 10 binary values. E.g., $x = (x_{ab}, x_{ac}, x_{ad}, x_{ae}, x_{bc}, x_{bd}, x_{be}, x_{cd}, x_{ce}, x_{de})$.
 (2) The loss function is the sum of weights from the selected edges. Formally, let $x(a, b)$ be the feature value (selected or not) of edge (a,b), then the loss function is $\mathcal{L}(x) = \sum_{a,b \in S} c(a, b) \cdot x(a, b)$.
2. Suppose the initial solution is (a,b), (b,c), (c,d), (d, e). The cost is 24. Note that you may start from a different solution. The neighbors of the current solution should remain a valid solution to preserve connectivity. So, we can define the neighborhood by swapping an selected edge with an unselected edge, while keeping the connectivity. Below is the process of the first 3 iterations:
 - Iteration 1: If we remove (a,b), we can add one of (a, c), (a, d), (a, e) to keep connectivity. But none of them is better than the current solution. If we remove (b,c), we cannot find any edge with smaller cost. In this iteration, only when we replacing (c, d) and add (a, e), the total cost reduces to 23. Thus, we select the neighbor solution (a,b), (b,c), (d, e), (a, e).
 - Iteration 2: no edge swapping operation can improve the cost, thus the algorithm returns (a,b), (b,c), (d, e), (a, e).

You may have different results if you choose a different initial solution.

Note: If the guarantee of valid solution is not mandatory, another solution is to allow invalid solutions to be candidate, but define a loss function in which a large penalty is assigned to the invalid solutions. Then, you could start from any initial solutions (not necessarily to be valid). The local search algorithm will tend to select valid solutions rather than invalid ones. But this method does not guarantee returning a valid solution.

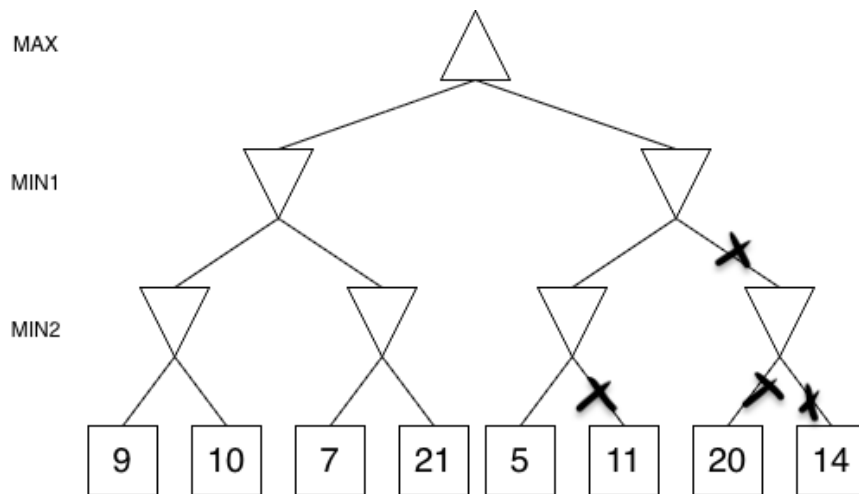
2. Adversarial Search (1.5 pts)

Consider a game tree with one MAX node and two MIN nodes below.

1. Report the values of the non-terminal nodes.
2. If alpha-beta pruning is applied, which edges would be pruned? Place cross ("x") on the pruned edges.

Solution:

1.
 - MIN2 nodes (from left to right): 9, 7, 5, 14
 - MIN1 nodes (from left to right): 7, 5
 - MAX node: 7
2. The edges to be pruned:



3. Genetic Algorithm (2 pts)

The 0-1 knapsack problem involves a list of n items, where each item has a positive reward and a weight in kg. Suppose you hold a bag with total capacity 14 kg. Your goal is to put items from the list into your bag so that the total reward of the selected item is maximised, while respecting the capacity of the bag. Each item can only be selected for once.

Item ID	Weight	Reward(\$)
1	2	1
2	2	2
3	3	6
4	5	8
5	12	10

1. Consider this problem as an evolutionary search problem. Define the representation of chromosomes and the fitness function. For simplicity, crossover/mutation operations are not required to produce a feasible solution (≤ 14 kg).
2. Assume the population is 4. Simulate one generation of genetic algorithm in solving the above problem. You should use example chromosomes to demonstrate the selection, crossover and mutation operations.

Solution:

1. The solution can be represented by a length-5 string, each position of the string takes 0 or 1 to denote whether to select an item or not. The fitness function can be defined as a weighted sum of the rewards:

$$fitness(s) = \begin{cases} \sum_{i=1}^5 s[i] \cdot Reward[i], & \sum_{i=1}^5 s[i] \cdot Weight[i] \leq 14 \\ 0, & Otherwise \end{cases}$$

2. • Random Initialization of 4 candidates: Total fitness = $9 + 0 + 9 + 12 = 30$.

Table 1: Initial chromosomes

Chromosome	Representation	Weight (kg)	Fitness
C1	1 1 1 0 0	7	9
C2	0 0 0 1 1	17	0
C3	1 0 0 1 0	7	9
C4	0 1 0 0 1	14	12

- Selection probabilities:

$$P(C1) = \frac{9}{30} = 0.3, \quad P(C2) = 0, \quad P(C3) = 0.3, \quad P(C4) = 0.4.$$

Suppose four parents are selected. Example outcome:

- Parent 1: C4 (fitness 12)
- Parent 2: C1 (fitness 9)
- Parent 3: C4 (fitness 12)
- Parent 4: C3 (fitness 9)

- Crossover of C4 and C1, say crossing site is 2:

$$(01|001), (11|100) \rightarrow (01|100)(11|001)$$

- Child 1 (01100): weight=5 kg, reward=8, fitness=8.
- Child 2 (11001): weight=16 kg, fitness=0.

Crossover of C4 and C3, say crossing site is 3:

$$(010|01), (100|10) \rightarrow (010|10)(100|01)$$

- Child 3 (01010): weight=7 kg, reward=10, fitness=10.
- Child 4 (10001): weight=14 kg, reward=11, fitness=11.
- Mutation: Randomly flip the third digit of Child 3, i.e., (01010) becomes (01110), weight=10 kg, reward=16, fitness=16.
- The new generation is:

Table 2: New generation

Chromosome	Representation	Weight (kg)	Fitness
C1	0 1 1 0 0	5	8
C2	1 1 0 0 1	16	0
C3	0 1 1 1 0	10	16
C4	1 0 0 0 1	14	11